



Actividad: Integración de patrones

Objetivo

Diseñar la arquitectura de clases y una implementación básica que use correctamente los siguientes patrones de diseño: *Bridge*, *Builder*, *Chain of responsibility*, *Flyweight*, *Interpreter* y *Mediator*.

Descripción

1. Contexto del problema

Una empresa de tecnología documental requiere desarrollar un **Motor de Documentos Inteligentes** capaz de importar, analizar, transformar y renderizar documentos masivos (como contratos, facturas o reportes financieros) en múltiples formatos de salida (PDF, HTML, Markdown).

El sistema debe ser altamente dinámico, extensible y eficiente en el uso de memoria, ya que procesará miles de páginas simultáneamente con miles de elementos repetidos (caracteres, iconos, estilos).

2. Requisitos

Para resolver el problema, la arquitectura debe integrar obligatoriamente los siguientes 6 patrones de diseño:

A. Pattern Flyweight (Gestión eficiente de memoria)

Objetivo: Optimizar la representación de los caracteres, símbolos e iconos del documento.

Aplicación: El documento contiene varios de elementos visuales. Debes separar el estado intrínseco (el carácter, la tipografía base, la imagen del icono) del estado extrínseco (posición X/Y en la página, color dinámico, tamaño/escala). El patrón Flyweight administrará y reutilizará las instancias compartidas de estos elementos.

B. Pattern Builder (Construcción compleja del documento)

Objetivo: Construir la estructura jerárquica del documento paso a paso.

Aplicación: Implementa un `DocumentBuilder` con métodos como `addHeader()`, `addParagraph()`, `addTable()`, `addFooter()` y `build()`. Esto permitirá ensamblar documentos complejos con diferentes configuraciones (por ejemplo, `ReporteEjecutivoBuilder` o `FacturaSimpleBuilder`) de forma limpia e independiente de la representación final.

C. Pattern Bridge (Desacoplamiento de estructura y renderizado)

Objetivo: Separar la abstracción del documento del mecanismo de renderizado/exportación.

Aplicación:

- Jerarquía de Abstracción: `Documento` (y sus variantes como `DocumentoPaginado`, `DocumentoContinuo`).
- Jerarquía de Implementación: `RenderizadorEngine` (con implementaciones concretas como `PdfRenderEngine`, `HtmlRenderEngine`, `MarkdownRenderEngine`).

Esto permite agregar nuevos tipos de documentos o nuevos formatos de exportación sin alterar la otra jerarquía.

D. Pattern Chain of Responsibility (Flujo de validaciones y transformaciones)

Objetivo: Procesar el contenido del documento a través de una tubería (pipeline) configurable.

Aplicación: Antes de renderizar el documento, este debe pasar por una cadena de manejadores (`ProcesadorHandler`):

- `ValidadorSintaxis`: Verifica que el documento no tenga etiquetas corruptas.
- `FiltroPalabrasProhibidas` / `Sanitizador`: Oculta información sensible o palabras no permitidas.
- `EvaluadorExpresiones`: Procesa las reglas o fórmulas internas (usando `Interpreter`).
- Si un manejador detecta un error crítico, interrumpe la cadena.

E. Pattern Interpreter (Evaluación de reglas y fórmulas)

Objetivo: Evaluar expresiones dinámicas embebidas en el documento.

Aplicación: Los documentos contienen marcadores con fórmulas dinámicas, por ejemplo: `#{PRECIO_BASE * 1.19 - DESCUENTO}` o variables como `#{FECHA_ACTUAL}`. Diseña una gramática simple con clases `ExpresionTerminal` (variables, números) y `ExpresionNoTerminal` (operaciones suma, multiplicación) para interpretar y calcular estos valores en tiempo de ejecución.

F. Pattern Mediator (coordinación de componentes de la interfaz/consola de control)

Objetivo: Reducir el acoplamiento entre los componentes del panel de control de procesamiento.

Aplicación: Implementa un `DocumentEditorMediator` que coordine la interacción entre la interfaz (o módulos) del usuario: el `SelectorDeFormato`, la `BarraDeHerramientasBuilder`, la `VistaPrevia` y el `BotonExportar`. Ninguno de estos componentes debe comunicarse directamente entre sí; todo evento (ej. "el usuario cambió el formato de salida a PDF") pasa por el Mediator para reconfigurar el Builder y el Renderizador correspondiente.

Entregable

Suba en un repositorio de GitHub lo siguiente:

Diagrama de Clases UML: Un diagrama integral que muestre claramente la relación e interacción entre los 6 patrones.

Código Fuente: Implementación funcional en Java

Casos de Prueba / Demo: Un programa principal (Main) que ejecute un flujo completo:

- Edición/Configuración vía Mediator.
- Creación del documento vía Builder utilizando Flyweights para los elementos repetidos.
- Procesamiento del documento a través de la Chain of Responsibility.
- Evaluación de al menos una fórmula matemática/dinámica mediante el Interpreter.

- Renderizado final del documento en al menos 2 formatos diferentes mediante el Bridge.